Fundamentos Computacionales en Biociencias

***Tema 1**. El sistema operativo. Entorno Unix/Linux. Práctica 1*

Grado en Biotecnología – Certificado Extensión Universitaria en Bioinformática y Ciencia de Datos
Universidad Católica de Valencia – Facultad de Veterinaria y Ciencias Experimentales
***Profesor:** Jose Vicente Roig Genovés jv.roig@ucv.es*
Curso académico 2026/2027 – **Grupo:** 1BIOINFOR



Universidad
Católica de
Valencia
San Vicente Mártir

Instrucciones generales

La práctica es evaluable y corresponde a un 15% de la nota final de la asignatura. Es necesario obtener al menos un 5 para poder promediar con el resto de las prácticas.

Se evaluarán los siguientes parámetros:

- Funcionamiento y adecuación de los comandos propuestos en cada apartado.
- Estructura final de los directorios y archivos en GitHub.
- Respuestas de redacción.

Trabaja de forma individual. El repositorio de GitHub que entregues será un reflejo tu propio trabajo.

Sigue el orden de las partes de la práctica (A → D): cada bloque se apoya en el anterior.

Ante cualquier duda de sintaxis, recuerda que puedes (y debes) consultar `man [comando]`.

El acceso al sistema Linux es decisión del alumno, podrá utilizar cualquiera de las opciones disponibles en la asignatura.

En las diferentes secciones de la práctica se realizarán preguntas de dos tipos (I) *comando* y (II) *redacción*. En las preguntas de tipo *comando*, encontrarás un espacio para escribir tu respuesta con la siguiente estructura:

Comando:
Output terminal:

En el apartado ***comando*** deberás incluir el comando utilizado en la *Shell.* En el apartado ***output terminal*** deberás copiar el *output* que genere la *Shell* al ejecutar el comando propuesto.

Si en la pregunta no aparece el apartado ***output terminal*** no será necesario incluirlo.

A modo de ejemplo:

1. Demuestra el directorio en el que te encuentras

Comando: `pwd`
Output terminal: `/home/fcb_alumno14`

En las preguntas tipo *redacción* tendrás un espacio disponible para redactar la respuesta, no hay límite de espacio para tu respuesta.

jv.roig@ucv.es

Parte A — Verificación del entorno y primera exploración

Empezamos accediendo al entorno Unix/Linux de una de las 3 formas que hemos aprendido y tenemos disponibles en *CEU Bioinformática y Ciencia de Datos*:

- **Conexión remota:** mediante SSH al servidor de la UCV. **RECOMENDADO**
- **Máquina virtual:** A través de tu ordenador, accede a Linux mediante la VirtualBox.
- **macOS:** desde la terminal en tu ordenador macOS.

A.1 — Identificación

Una vez estemos en el entorno Unix/Linux:

1. **Ejecuta el comando que muestra tu nombre de usuario actual. Anota el resultado.**

Comando: `whoami`

Output terminal: `fcb_alumno14`

2. **Ejecuta el comando que muestra el directorio de trabajo actual. ¿En qué directorio te encuentras justo después de conectar? ¿Coincide con tu directorio *home*?**

Comando: pwd

Output terminal: `/home/fcb_alumno14`

A.2 — Explorando tu directorio *home* y material de la práctica.

Si estás trabajando desde el servidor de la UCV, encontrarás la carpeta `fcb_material_practica1` en tu directorio `~/`.
Si estás trabajando desde la máquina virtual o desde tu equipo macOS, deberás descargarte la carpeta `fcb_material_practica1` desde el campus docencia y moverla al directorio donde estés trabajando.

El contenido y estructura del directorio es el siguiente:

```
fcb_material_practica1/
      ├── README.txt
      ├── datos_muestras.txt
      ├── secuencias_DNA.fasta
      ├── analisis_previos/
           └── resumen_2024.txt
           └── notas_control_calidad.txt
```

3. **Lista el contenido de la carpeta `fcb_material_practica1` incluyendo los archivos ocultos y con formato detallado y tamaños legibles.**

Comando: `cd fcb_material_practica1 y despúes ls`

Output terminal: datos_muestras.txt README.txt secuencias_DNA.fasta
fcb_alumno14@UCVVBox:~/fcb_material_practica1$

Pregunta A.2.1 — ¿Qué es, en Unix/Linux, un archivo o directorio "oculto"? ¿Cómo se identifican por su nombre? Da un ejemplo de alguno que hayas visto en tu propio *home*.

```
Cuando un archivo o directorio es oculto, empieza la denominación con un .,
este recurso sirve para no ser eliminados inconscientemente entre otras cosas.
```

Pregunta A.2.2 — Compara la salida de `ls -a` con la de `ls -lh`. ¿Qué diferencia exactamente introduce la opción -h? Consulta `man ls` si lo necesitas.

```
La –a: datos_muestras.txt  README.txt  secuencias_DNA.fasta
fcb_alumno14@UCVVBox:~/fcb_material_practica1$

Ls –lh: total 16K
drwxr-xr-x 2 fcb_alumno14 fcb_alumno14 4,0K sep 27 22:14 analisis_previos
-rw-r--r-- 1 fcb_alumno14 fcb_alumno14 2,2K sep 27 22:14 datos_muestras.txt
-rw-r--r-- 1 fcb_alumno14 fcb_alumno14  611 sep 27 22:14 README.txt
-rw-r--r-- 1 fcb_alumno14 fcb_alumno14 1,3K sep 27 22:14 secuencias_DNA.fasta
fcb_alumno14@UCVVBox:~/fcb_material_practica1$
```

Pregunta A.2.3 — Ordena los archivos presentes en `fcb_material_practica1` y `fcb_material_practica1/analisis_previos` de mayor a menor tamaño.

fcb_alumno14@UCVVBox:~$ find ~/fcb_material_practica1 -type f -exec du -h {} + | sort -hr

fcb_alumno14@UCVVBox:~$ find ~/fcb_material_practica1/analisis_previos/ -type f -exec du -h {} + | sort –hr

4,0K /home/fcb_alumno14/fcb_material_practica1/analisis_previos/resumen_2024.txt

4,0K /home/fcb_alumno14/fcb_material_practica1/analisis_previos/notas_control_calidad.txt

Antes de continuar, muestra el contenido del archivo `README.txt`

Comando:

Output terminal: `cat fcb_material_practica1/README.txt`

```
  - datos_muestras.txt        -> tabla de muestras de laboratorio (ejemplo)
  - secuencias_DNA.fasta      -> secuencias ADN
  - analisis_previos/

Esta carpeta contiene archivos de ejemplo para que trabajar con  los comandos
de exploracion de contenido vistos en clase: cat, less, head, tail, wc,
grep, find...
```

A.3 — Uso de la ayuda del sistema

4. Consulta la página de manual del comando `ls`.

Comando: `man ls`

Pregunta A.3.1 — Busca en el manual una opción de `ls` que NO se haya visto en clase (distinta de -a, -l, -h) y explica en una frase qué hace.

```
-t, ordena los archivos según su orden de última modificación. No sabía que
ordenaba por modificación sino por orden de creación.
```

5. Sal del manual y revisa el historial de comandos que llevas ejecutados en esta sesión.

Comando: `History`

6. Guarda en un archivo (.txt) todo el historial de comandos que has utilizado hasta ahora. (*Ojo con el nombre…*)

Comando: `history > historial_COMANDOS.txt`

Parte B — Construcción del espacio de trabajo

Crea, desde cero, la estructura de un pequeño proyecto de bioinformática. Esta estructura será la base de la entrega final en GitHub.

B.1 — Árbol de directorios del proyecto

Sitúate en tu directorio *home* (~) y crea la siguiente estructura de directorios:

```
proyecto_biotec/
├── datos_crudos/
├── datos_procesados/
├── scripts/
├── resultados/
└── documentacion/
```

Requisito — Debes construir esta estructura combinando al menos: `mkdir -p`, navegación con `cd` usando ruta relativa y ruta absoluta al menos una vez cada una, y una verificación final con `ls`. No se aceptará como válido crear todos los directorios desde el *home* sin haber cambiado de directorio en ningún momento.

7. Crea el directorio raíz del proyecto desde tu *home*.

```
Comando: mkdir proyecto_informatica
```

8. Entra en él usando una ruta relativa, y desde dentro crea de una sola vez los cinco subdirectorios usando `mkdir -p`.

```
Comando 1: cd proyecto_informatica
```

```
Comando 2: mkdir -p datoscrudos datos_procesados scripts resultados documentacion
```

9. Vuelve a tu directorio *home* usando la ruta absoluta.

```
Comando: cd y posteriormente home
```

10. Desde el *home*, verifica el árbol creado listando el contenido de `proyecto_biotec` indicando la ruta absoluta completa como argumento de `ls`.

```
Comando: cd proyecto_informatica y posteriormente ls -l
```

B.2 — Corrección de nombres mal formados

Un compañero te ha pasado el nombre que debería tener un archivo de resultados: "*mis resultados finales.txt*". Este nombre incumple las convenciones de nomenclatura de Unix vistas en clase.

Pregunta B.2.1 — ¿Por qué es problemático usar espacios en nombres de archivo en Unix/Linux? ¿Qué alternativas viste en clase para nombrar correctamente este archivo?

```
Debido a que la shell, reconoce los espacios como separadores de argumentos de un comando. Pudiendo provocar errores al ejecutar ordenes sobre esos archivos. Por lo tanto, se remplazarán los espacios por guiones bajos.
```

11. **Crea el archivo vacío con el nombre corregido (sin espacios) directamente dentro de `resultados/`.**

```
Comando: touch mis_resultados_finales.txt (previamente habiendo entrado en la carpeta resultados)
```

12. **Ahora, a propósito, crea también un archivo con el nombre problemático (con espacios) escapando los espacios con barra invertida(\), para comprobar que también es posible, y bórralo justo después con `rm` (deberás escapar los espacios igualmente para poder borrarlo).**

```
Comando 1: touch mis\ resultados\ finales.txt
```

```
Comando 2: rm mis\ resultados\ finales.txt
```

B.3 — Creación de archivos

13. **Crea, con touch un archivo vacío dentro de cada uno de los otros cuatro subdirectorios (`datos_crudos/`, `datos_procesados/`, `scripts/` y `documentacion/`), con un nombre descriptivo y correcto según las convenciones vistas (p.Ej: notas.txt, analisis.sh, etc.). No hace falta que tengan contenido.**

```
Comando 1: touch datoscrudos/datos_originales.txt (se ha de utilizart el comando "cd" antes de crear el archivo .txt con el touch.
```

```
Comando 2: touch datosprocesados/datos.txt
```

```
Comando 3: touch script/analisis.sh
```

```
Comando 4: touch documentacion/notas.sh
```

B.4 — Tu primer archivo con contenido real

14. **Dentro de `documentacion/` crea un archivo con contenido real (no vacío) que resuma, en tus propias palabras, tres conceptos clave del Tema 1.**

Pista: `cat` con redirección para escribir directamente el contenido.

```
Comando: Empleas el comando "cd" para acceder al directorio "anotaciones" y
una vez dentro utilizas el comando "cat" para poder escribir en el archivo
.txt
```

El texto debe responder, en un párrafo cada una y con tus propias palabras, a estas tres cuestiones:

- **¿Qué es el kernel de un sistema operativo y qué función cumple?**
- **¿Qué diferencia hay entre *shell* y terminal?**
- **¿Por qué Unix/Linux es especialmente relevante en el ámbito de la bioinformática y la computación científica, frente a otros sistemas operativos?**

15. **Genera un documento (.txt) con el historial de comandos para este ejercicio. Guárdalo en la carpeta `proyecto_biotec/scripts/` con el nombre `historial_comandos_B.txt`**

```
Comando: Una vez dentro de la carpeta "scripts" empleamos el comando hisroty
seguido de un > para crear el archivo .txt
```

Parte C — Manipulación y exploración de archivos

En esta parte vas a incorporar a tu proyecto el material de apoyo (`fcb_material_practica1/`) que ya tienes en tu directorio *home*, y vas a extraer información real de él.

C.1 — Incorporar el material a tu proyecto

16. **Copia, de forma recursiva, la carpeta completa `fcb_aterial_practica1/` dentro de `datos_crudos/` de tu proyecto.**

Comando: `cp -r ~/fcb_material_practica1 ~/proyecto_informatica/datos_crudos/`

17. **Verifica que la copia se ha realizado correctamente, listando el contenido de `datos_crudos/`.**

Comando: `ls a la carpeta datos_crudos`

18. **Entra en la copia que acabas de hacer: a partir de aquí trabajarás con rutas relativas cortas, ya situado dentro de la carpeta.**

Comando: `empleando un cd completo, es decir, no introducir el comando cd para cada directorio/carpeta, sino que emplear uno solo e ir entrando a las carpetas a partir de la /.`

C.2 — Inspección de contenido

19. **Muestra el contenido completo de `datos_muestras.txt` con `cat`.**

Comando: `cd para entrar en fcb_material.. Y despues emplear el cat.`

20. **Muestra únicamente las 5 primeras líneas y, después, las 5 últimas.**

Comando 1: `head -n 5 datos_muestras.txt`

```
id_muestra      especie tipo_muestra    concentracion_ng_ul     estado
M001    Saccharomyces_cerevisiae        ARN     153.8   control
M002    Escherichia_coli        ADN     240.2   tratamiento
M003    Mus_musculus    ADN     152.7   control
M004    Homo_sapiens_cultivo    proteina        226.2   tratamiento
```

```
Comando 2: tail -n 5 datos_muestras.txt
M045    Drosophila_melanogaster ARN     139.2   tratamiento
M046    Escherichia_coli        ADN     280.1   control
M047    Drosophila_melanogaster ADN     353.2   control
M048    Escherichia_coli        ADN     432.5   control
M049    Danio_rerio     ARN     51.4    tratamiento
```

Pregunta C.2.1 — Fíjate en la primera línea que muestra `head`. ¿Qué contiene exactamente? ¿Es una muestra de laboratorio como las demás, o es otra cosa?

```
Se trata del encabezamiento de una tabla, donde contiene el tipo de
información que se proporcionara en cada columna.
```

21. **Cuenta cuántas líneas tiene el archivo en total. (pista: `man(wc)`)**

```
Comando: wc -l
```

```
Output terminal: 50 datos_muestras
```

Pregunta C.2.2 — ¿Cuántas líneas ha contado? Relaciona tu respuesta con lo observado en la Pregunta C.2.1: El comando que has usado, ¿"entiende" el significado de lo que hay escrito en el archivo, o simplemente cuenta líneas de texto sin más? ¿Coincide entonces ese número con el número real de muestras de laboratorio?

```
Ha contado 50 líneas en total, no entiende lo que hay escrito únicamente
cuenta las líneas, de esta manera no coincide con las pruebas realizadas,
porque, por ejemplo, la primera línea no se trata de una prueba.
```

C.3 — Búsqueda de contenido con `grep`

22. **Ejecuta el comando `grep -c control datos_muestras.txt`. Explica cómo funciona el comando `grep` y cuál ha sido el resultado obtenido con el comando ejecutado.**

```
Output terminal: 29
```

Grep quiere decir buscar coincidencias, -c le indica contar el número de líneas que coinciden con la palabra que hemos realizado "control" dentro del archivo .txt

23. Busca ahora todas las muestras de una especie concreta, por ejemplo *Escherichia_coli*.

Comando: grep Escherichia*coil datos*_m0uestras.txt

```
M002    Escherichia_coli        ADN     240.2   tratamiento
M005    Escherichia_coli        ADN     171.8   control
M008    Escherichia_coli        ARN     61.0    control
M012    Escherichia_coli        ADN     20.6    control
M014    Escherichia_coli        ARN     305.4   control
M020    Escherichia_coli        ADN     163.5   tratamiento
M027    Escherichia_coli        ARN     291.8   control
M032    Escherichia_coli        ARN     125.0   tratamiento
M043    Escherichia_coli        ARN     312.8   tratamiento
M046    Escherichia_coli        ADN     280.1   control
M048    Escherichia_coli        ADN     432.5   control
```

Pregunta C.3.1 — Anota cuántas muestras de *Escherichia*_coli ha encontrado `grep`. Repite la búsqueda con otra especie distinta que veas en el archivo y anota también su recuento.

```
11
```

Comando: grep -c Saccaromyces_cerevisae datos_muestras.txt

```
0
```

Bonus (opcional): en bioinformática, cada secuencia de un archivo FASTA empieza por una línea de cabecera que arranca con el símbolo >. Sin necesidad de abrir el archivo completo, usa `grep` para contar cuántas secuencias contiene secuencias_DNA.fasta.

Comando: grep -c '^>' secuencias_DNA.fasta

Output terminal: 16

C.4 — `rm` vs. `rmdir`

24. Crea intencionadamente un directorio de prueba llamado `temp_borrar` dentro de `proyecto_biotec`, y dentro de él un archivo vacío cualquiera con `touch`.

```
Comando 1: cd proyecto_informatica - mkdir borrador
```

```
Comando 2: touch prueba.txt
```

25. Intenta borrar `temp_borrar` con `rmdir`. Observa y anota el mensaje de error.

```
Comando: rmdir
```

```
Output terminal: el directorio no está vacio
```

Pregunta C.5.1 — ¿Por qué falla `rmdir` en este caso? ¿Qué condición exige `rmdir` para poder borrar un directorio?

```
El comando rmdir falla porque el directorio borrador contiene un archivo en su interior. Este comando únicamente permite eliminar directorios vacíos. Para eliminar un directorio que contiene archivos o subdirectorios es necesario utilizar rm -r de manera que se autoriza la eliminación del archivo.
```

26. Borra correctamente ahora el directorio `temp_borrar` y todo su contenido con una única orden.

```
Comando:  rm -r temp_borrar
```

27. Genera un documento (.txt) con el historial de comandos para este ejercicio. Guárdalo en la carpeta `proyecto_biotec/scripts/` con el nombre `historial_comandos_C.txt`

```
Comando: history > historial_comandos-C.txt
```

Parte D — Entrega en GitHub y memoria final

GitHub es una plataforma en la nube para alojar repositorios (proyectos versionados). La usaremos, de momento, únicamente a través de su interfaz web, sin necesidad de comandos (de momento…). Aquí la usamos como repositorio de entrega y como primera piedra de tu portfolio de bioinformática creado en el CEU de Bioinformática y Ciencia de Datos.

D.1 — Crear tu cuenta

28. **Si todavía no tienes cuenta, ve a github.com y crea una cuenta gratuita. Usa un nombre de usuario profesional y reconocible.**

D.2 — Crear el repositorio

29. **Crea un nuevo repositorio (botón "*New repository*").**
30. **Nómbralo de forma clara, por ejemplo: certificado-bioinformatica-ciencia-datos o fundamentos-computacion-biociencias.**
31. **Márcalo como público y añade una breve descripción de qué es este repositorio y para qué asignatura es.**
32. **Marca la opción de inicializar el repositorio con un README.**

D.3 — Subir tu proyecto

Desde la página de tu repositorio, usa la opción "*Add file → Upload files*" y arrastra o selecciona la carpeta completa `proyecto_biotec` (con toda su estructura de subcarpetas y archivos construida en la práctica).

D.4 — Memoria final (editar el README.md)

Edita el archivo README.md de tu repositorio directamente desde la interfaz web de GitHub (icono de lápiz) y redacta una explicación de la carpeta y contenido que acabas de subir.

D.5 — Entrega

- Enlace a repositorio Github con archivo README.md y carpeta `proyecto_biotec` donde estén todos los directorios y archivos de la práctica.
- Documento pdf con comandos y respuestas a las preguntas planteadas en la práctica 1. Plataforma UCV.

```
Link a GitHub:
```